

Module 8.2 - Event Handling

Learning Objectives: Master custom event binding and emission patterns, understand the differences between EventEmitter, Subject, and Observable for data flow, implement complex event handling scenarios, and apply best practices for event-driven component communication.

1. Understanding Event Handling in Angular

1.1 What is Event Handling?

Event handling in Angular refers to the mechanism of responding to user actions (clicks, inputs, etc.) and custom component events. Angular provides several ways to handle events, from simple DOM events to complex custom event systems.

Types of Events in Angular:

- **DOM Events:** Native browser events (click, input, keyup, etc.)
- **Custom Events:** Component-specific events using EventEmitter
- **Observable Events:** RxJS-based event streams
- **Subject-based Events:** Multicast event broadcasting

1.2 Event Binding Syntax

Syntax	Description	Example
<code>(event)</code>	Event binding	<code>(click)="handler()"</code>
<code>\$event</code>	Event payload	<code>(click)="handler(\$event)"</code>
<code>\$any(\$event)</code>	Type casting	<code>(input)="update(\$any(\$event).target.value)"</code>
<code>#ref</code>	Template reference	<code>#input (keyup)="log(input.value)"</code>

2. Custom Event Binding with EventEmitter

2.1 EventEmitter Fundamentals

`EventEmitter` is Angular's built-in class for emitting custom events from components. It extends `RxJS Subject` and is used with the `@Output` decorator.

Key Features:

- Type-safe event emission
- Synchronous by default
- Can emit any data type
- Automatically managed by Angular

2.2 Basic EventEmitter Example

Child Component (`notification.component.ts`):

```
import { Component, Output, EventEmitter } from '@angular/core';

interface NotificationEvent {
  type: 'info' | 'success' | 'warning' | 'error';
  message: string;
  timestamp: Date;
}

@Component({
  selector: 'app-notification',
  template: `
    <div class="notification-panel">
      <h3>Send Notification</h3>

      <select [(ngModel)]="selectedType">
        <option value="info">Info</option>
        <option value="success">Success</option>
        <option value="warning">Warning</option>
        <option value="error">Error</option>
      </select>

      <input [(ngModel)]="message" placeholder="Enter message">
    </div>
  `
})
```

```

    <button (click)="sendNotification()">Send</button>
  </div>
  `
})
export class NotificationComponent {
  selectedType: 'info' | 'success' | 'warning' | 'error' = 'info';
  message: string = '';

  // EventEmitter with typed payload
  @Output() notificationSent = new EventEmitter<NotificationEvent>();

  sendNotification() {
    if (this.message.trim()) {
      const event: NotificationEvent = {
        type: this.selectedType,
        message: this.message,
        timestamp: new Date()
      };

      // Emit the event
      this.notificationSent.emit(event);

      // Clear the form
      this.message = '';
    }
  }
}

```

Parent Component:

```

export class DashboardComponent {
  notifications: NotificationEvent[] = [];

  onNotificationSent(event: NotificationEvent) {
    console.log('Notification received:', event);
    this.notifications.push(event);

    // Show system notification based on type
    this.displayNotification(event);
  }

  displayNotification(event: NotificationEvent) {
    const colors = {
      info: '#17a2b8',
      success: '#28a745',

```

```

    warning: '#ffc107',
    error: '#dc3545'
  };

  console.log(`%c${event.type.toUpperCase()}: ${event.message}`,
    `color: ${colors[event.type]}; font-weight: bold;`
  );
}
}

```

Parent Template:

```

<h2>Dashboard</h2>

<app-notification (notificationSent)="onNotificationSent($event)">
</app-notification>

<h3>Notification History</h3>
<div class="notification-list">
  <div *ngFor="let notif of notifications"
    [class]='notification-item ' + notif.type">
    <strong>{{ notif.type | uppercase }}</strong>
    <p>{{ notif.message }}</p>
    <small>{{ notif.timestamp | date:'short' }}</small>
  </div>
</div>

```

Dashboard

Send Notification

[Type: Info ▼]

System update available

[Send Button]

Notification History

INFO

System update available

1/30/26, 2:30 PM

SUCCESS

Data saved successfully

1/30/26, 2:31 PM

Console Output:
INFO: System update available

SUCCESS: Data saved successfully

2.3 Multiple Event Emitters

Components can have multiple EventEmitters for different event types.

Child Component (file-uploader.component.ts):

```
import { Component, Output, EventEmitter } from '@angular/core';

interface UploadProgress {
  fileName: string;
  progress: number;
}

interface UploadResult {
  fileName: string;
  success: boolean;
  url?: string;
  error?: string;
}

@Component({
  selector: 'app-file-uploader',
  template: `
    <div class="file-uploader">
      <h3>File Upload</h3>

      <input type="file" (change)="onFileSelected($event)" #fileInput>

      <div *ngIf="uploading" class="progress-bar">
        <div class="progress" [style.width.%]="progress"></div>
        <span>{{ progress }}%</span>
      </div>
    </div>`
})
```

```

    </div>

    <button (click)="cancelUpload()" *ngIf="uploading">Cancel</button>
  </div>
  ,
})
export class FileUploaderComponent {
  // Multiple output events for different scenarios
  @Output() uploadStarted = new EventEmitter<string>();
  @Output() uploadProgress = new EventEmitter<UploadProgress>();
  @Output() uploadCompleted = new EventEmitter<UploadResult>();
  @Output() uploadCancelled = new EventEmitter<string>();
  @Output() uploadError = new EventEmitter<{fileName: string, error: string}>();

  uploading = false;
  progress = 0;
  currentFileName = '';

  onFileSelected(event: Event) {
    const input = event.target as HTMLInputElement;
    if (input.files && input.files.length > 0) {
      const file = input.files[0];
      this.startUpload(file);
    }
  }

  startUpload(file: File) {
    this.currentFileName = file.name;
    this.uploading = true;
    this.progress = 0;

    // Emit upload started
    this.uploadStarted.emit(file.name);

    // Simulate upload with progress updates
    const interval = setInterval(() => {
      this.progress += 10;

      // Emit progress updates
      this.uploadProgress.emit({
        fileName: file.name,
        progress: this.progress
      });

      if (this.progress >= 100) {
        clearInterval(interval);
        this.uploading = false;
      }
    }, 1000);
  }
}

```

```

    // Emit completion
    this.uploadCompleted.emit({
      fileName: file.name,
      success: true,
      url: `https://example.com/files/${file.name}`
    });
  }
}, 500);
}

cancelUpload() {
  this.uploading = false;
  this.progress = 0;

  // Emit cancellation
  this.uploadCancelled.emit(this.currentFileName);
}
}

```

Parent Component:

```

export class FileManagerComponent {
  uploadLog: string[] = [];

  onUploadStarted(fileName: string) {
    this.uploadLog.push(`Upload started: ${fileName}`);
    console.log('Upload started:', fileName);
  }

  onUploadProgress(progress: UploadProgress) {
    console.log(`${progress.fileName}: ${progress.progress}%`);
  }

  onUploadCompleted(result: UploadResult) {
    this.uploadLog.push(`✓ Upload completed: ${result.fileName}`);
    console.log('Upload completed:', result);
  }

  onUploadCancelled(fileName: string) {
    this.uploadLog.push(`✗ Upload cancelled: ${fileName}`);
    console.log('Upload cancelled:', fileName);
  }

  onUploadError(error: {fileName: string, error: string}) {

```

```

    this.uploadLog.push(`X Upload error: ${error.fileName} - ${error.error}`);
    console.error('Upload error:', error);
  }
}

```

Parent Template:

```

<h2>File Manager</h2>

<app-file-uploader
  (uploadStarted)="onUploadStarted($event)"
  (uploadProgress)="onUploadProgress($event)"
  (uploadCompleted)="onUploadCompleted($event)"
  (uploadCancelled)="onUploadCancelled($event)"
  (uploadError)="onUploadError($event)">
</app-file-uploader>

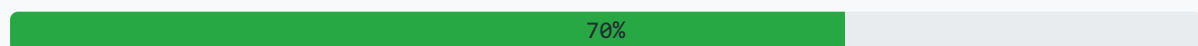
<h3>Upload Log</h3>
<ul>
  <li *ngFor="let log of uploadLog">{{ log }}</li>
</ul>

```

File Manager

File Upload

[Choose File: document.pdf]



[Cancel Button]

Upload Log

- Upload started: document.pdf
- ✓ Upload completed: document.pdf

Console Output:

```

Upload started: document.pdf
document.pdf: 10%
document.pdf: 20%

```



```
...  
document.pdf: 100%  
Upload completed: {fileName: "document.pdf", success: true, url: "https://..."}  
...
```

2.4 Event Bubbling and Event Object

Understanding the `$event` object and how to work with native DOM events.

Component TypeScript:

```
export class EventHandlingComponent {  
  // Handle different event types  
  handleClick(event: MouseEvent) {  
    console.log('Click event:', event);  
    console.log('Target element:', event.target);  
    console.log('Mouse position:', event.clientX, event.clientY);  
    console.log('Ctrl key pressed:', event.ctrlKey);  
  }  
  
  handleInput(event: Event) {  
    const input = event.target as HTMLInputElement;  
    console.log('Input value:', input.value);  
  }  
  
  handleKeyPress(event: KeyboardEvent) {  
    console.log('Key pressed:', event.key);  
    console.log('Key code:', event.code);  
  
    if (event.key === 'Enter') {  
      console.log('Enter key detected!');  
    }  
  
    if (event.ctrlKey && event.key === 's') {  
      event.preventDefault();  
      console.log('Save shortcut triggered');  
    }  
  }  
  
  handleSubmit(event: Event) {  
    event.preventDefault(); // Prevent form submission  
  }  
}
```

```

    console.log('Form submission prevented');
  }
}

```

Component Template:

```

<h3>Event Handling Examples</h3>

<!-- Click event with MouseEvent -->
<button (click)="handleClick($event)">Click Me</button>

<!-- Input event -->
<input (input)="handleInput($event)" placeholder="Type something">

<!-- Keyboard event -->
<input (keydown)="handleKeyPress($event)" placeholder="Press keys">

<!-- Form submit -->
<form (submit)="handleSubmit($event)">
  <input type="text" name="username">
  <button type="submit">Submit</button>
</form>

<!-- Multiple events on same element -->
<div
  (mouseenter)="log('Mouse entered')"
  (mouseleave)="log('Mouse left')"
  (click)="log('Clicked')"
  style="padding: 20px; border: 1px solid #ccc;">
  Hover and click me
</div>

```

Event Handling Examples

[Click Me Button]

Console Output (after clicking button):

Click event: MouseEvent {...}
Target element: <button>Click Me</button>
Mouse position: 250, 180
Ctrl key pressed: false

Console Output (after typing):

Input value: H
Input value: He
Input value: Hello

Console Output (after pressing Enter):

Key pressed: Enter
Key code: Enter
Enter key detected!

2.5 Event Filtering and Conditional Emission

Sometimes you only want to emit events under certain conditions.

Child Component:

```
export class SearchComponent {
  @Output() searchPerformed = new EventEmitter<string>();
  @Output() searchCleared = new EventEmitter<void>();

  searchTerm: string = '';
  minSearchLength: number = 3;

  onSearchChange() {
    // Only emit if search term is long enough
    if (this.searchTerm.length >= this.minSearchLength) {
      this.searchPerformed.emit(this.searchTerm);
    } else if (this.searchTerm.length === 0) {
      this.searchCleared.emit();
    }
  }

  onSearchKeyPress(event: KeyboardEvent) {
    // Only emit on Enter key
  }
}
```

```

    if (event.key === 'Enter' && this.searchTerm.trim()) {
      this.searchPerformed.emit(this.searchTerm);
    }
  }
}

```

Template:

```

<input
  [(ngModel)]="searchTerm"
  (ngModelChange)="onSearchChange()"
  (keydown)="onSearchKeyPress($event)"
  placeholder="Search (min 3 chars)">

```

3. EventEmitter vs Subject vs Observable

3.1 Understanding the Differences

Feature	EventEmitter	Subject	Observable
Purpose	Component events	Event bus, multicasting	Data streams
Multicast	Yes	Yes	No (cold), Yes (hot)
Can emit	Yes	Yes	No (producer only)
Can subscribe	Yes	Yes	Yes
Use with @Output	Yes (designed for)	No (shouldn't)	Yes (read-only)
Angular specific	Yes	No (RxJS)	No (RxJS)

Key Point: `EventEmitter` actually extends `RxJS Subject` internally, but it's designed specifically for Angular's `@Output` decorator and component events.

3.2 EventEmitter - Component Events Only

Best Use Case: @Output events in components.

```
export class ButtonComponent {
  // ☒ Correct usage: with @Output
  @Output() clicked = new EventEmitter<void>();

  handleClick() {
    this.clicked.emit();
  }
}

// ☒ Wrong usage: EventEmitter without @Output
export class DataService {
  // Don't do this! Use Subject instead
  dataChanged = new EventEmitter<any>();
}
```

3.3 Subject - Event Bus Pattern

Subject is an RxJS type that acts as both Observable and Observer. Perfect for creating event buses and shared state.

Types of Subjects:

- **Subject:** Basic subject, no initial value
- **BehaviorSubject:** Has initial value, emits last value to new subscribers
- **ReplaySubject:** Replays N previous values to new subscribers
- **AsyncSubject:** Emits only the last value when completed

Event Service with Subject (event-bus.service.ts):

```
import { Injectable } from '@angular/core';
import { Subject, Observable } from 'rxjs';
import { filter, map } from 'rxjs/operators';

interface AppEvent {
  type: string;
  payload?: any;
}
```

```
}

@Injectables({
  providedIn: 'root'
})
export class EventBusService {
  // Private Subject for internal use
  private eventSubject = new Subject<AppEvent>();

  // Public Observable for consumers (read-only)
  public events$: Observable<AppEvent> = this.eventSubject.asObservable();

  // Emit event
  emit(event: AppEvent) {
    this.eventSubject.next(event);
  }

  // Get events of specific type
  on(eventType: string): Observable<any> {
    return this.events$.pipe(
      filter(event => event.type === eventType),
      map(event => event.payload)
    );
  }
}
```

Component A - Emitting Events:

```
export class HeaderComponent {
  constructor(private eventBus: EventBusService) {}

  onLogout() {
    this.eventBus.emit({
      type: 'USER_LOGGED_OUT',
      payload: { userId: 123, timestamp: new Date() }
    });
  }

  onThemeChange(theme: string) {
    this.eventBus.emit({
      type: 'THEME_CHANGED',
      payload: { theme }
    });
  }
}
```

```
    });  
  }  
}
```

Component B - Subscribing to Events:

```
import { Component, OnInit, OnDestroy } from '@angular/core';  
import { Subscription } from 'rxjs';  
  
export class SidebarComponent implements OnInit, OnDestroy {  
  private subscriptions = new Subscription();  
  
  constructor(private eventBus: EventBusService) {}  
  
  ngOnInit() {  
    // Subscribe to specific event types  
    this.subscriptions.add(  
      this.eventBus.on('USER_LOGGED_OUT').subscribe(data => {  
        console.log('User logged out:', data);  
        this.handleLogout();  
      })  
    );  
  
    this.subscriptions.add(  
      this.eventBus.on('THEME_CHANGED').subscribe(data => {  
        console.log('Theme changed to:', data.theme);  
        this.applyTheme(data.theme);  
      })  
    );  
  }  
  
  ngOnDestroy() {  
    // Clean up subscriptions  
    this.subscriptions.unsubscribe();  
  }  
  
  handleLogout() {  
    // Handle logout logic  
  }  
  
  applyTheme(theme: string) {
```

```
// Apply theme logic
}
}
```

Event Flow:

Header Component: User clicks logout

→ EventBus emits: USER_LOGGED_OUT

Sidebar Component: Receives event

Console: User logged out: {userId: 123, timestamp: ...}

Header Component: User changes theme to "dark"

→ EventBus emits: THEME_CHANGED

Sidebar Component: Receives event

Console: Theme changed to: dark

3.4 BehaviorSubject - State Management

BehaviorSubject always has a current value and emits it immediately to new subscribers.

User State Service (user-state.service.ts):

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

interface User {
  id: number;
  name: string;
  email: string;
  role: string;
}

@Injectable({
  providedIn: 'root'
})
export class UserStateService {
  // Private BehaviorSubject with initial value (null = not logged in)
  private userSubject = new BehaviorSubject<User | null>(null);
```



```
// Public Observable for components to subscribe
public user$: Observable<User | null> = this.userSubject.asObservable();

// Get current user synchronously
get currentUser(): User | null {
  return this.userSubject.value;
}

// Set user
setUser(user: User) {
  this.userSubject.next(user);
}

// Clear user (logout)
clearUser() {
  this.userSubject.next(null);
}

// Check if user is logged in
isLoggedIn(): boolean {
  return this.userSubject.value !== null;
}
}
```

Header Component - Displaying User:

```
export class HeaderComponent implements OnInit {
  user$: Observable<User | null>;

  constructor(private userState: UserStateService) {
    this.user$ = this.userState.user$;
  }

  ngOnInit() {
    // Component automatically receives current user value
    // and all future updates
  }

  logout() {
    this.userState.clearUser();
  }
}
```

Header Template:

```
<nav class="header">
  <div *ngIf="user$ | async as user; else notLoggedIn">
    <span>Welcome, {{ user.name }}</span>
    <button (click)="logout()">Logout</button>
  </div>

  <ng-template #notLoggedIn>
    <button routerLink="/login">Login</button>
  </ng-template>
</nav>
```

Login Component - Updating User:

```
export class LoginComponent {
  constructor(private userState: UserStateService) {}

  login(username: string, password: string) {
    // Simulate authentication
    const user: User = {
      id: 1,
      name: 'John Doe',
      email: 'john@example.com',
      role: 'admin'
    };

    // Update user state - all subscribers receive update automatically
    this.userState.setUser(user);
  }
}
```

Before Login:

[Login Button]

After Login:

Welcome, John Doe [Logout Button]

Key Benefit: When LoginComponent updates the user, HeaderComponent (and any other component subscribing to user\$) automatically receives the update without direct component communication!

3.5 ReplaySubject - Historical Events

ReplaySubject replays a specified number of previous emissions to new subscribers.

Notification Service:

```
import { Injectable } from '@angular/core';
import { ReplaySubject, Observable } from 'rxjs';

interface Notification {
  id: string;
  message: string;
  type: 'info' | 'success' | 'error';
  timestamp: Date;
}

@Injectable({
  providedIn: 'root'
})
export class NotificationService {
  // ReplaySubject that replays last 5 notifications to new subscribers
  private notificationSubject = new ReplaySubject<Notification>(5);

  public notifications$: Observable<Notification> =
    this.notificationSubject.asObservable();

  showNotification(message: string, type: 'info' | 'success' | 'error' = 'info') {
    const notification: Notification = {
      id: Date.now().toString(),
      message,
      type,
      timestamp: new Date()
    };

    this.notificationSubject.next(notification);
  }
}
```

Usage:

```
export class DashboardComponent implements OnInit {
  constructor(private notificationService: NotificationService) {}
}
```

```
ngOnInit() {  
  // This component will receive the last 5 notifications  
  // even if they were emitted before this component subscribed  
  this.notificationService.notifications$.subscribe(notif => {  
    console.log('Notification:', notif);  
    this.displayNotification(notif);  
  });  
}  
  
displayNotification(notif: Notification) {  
  // Display logic  
}  
}
```

3.6 When to Use Each

Use EventEmitter when:

- Working with @Output decorator in components
- Parent-child component communication
- Emitting DOM-like events from components

Use Subject when:

- Creating an event bus for unrelated components
- Need to manually emit values from multiple places
- Don't need to store/replay previous values

Use BehaviorSubject when:

- Managing application state
- Need to access current value synchronously
- New subscribers should receive the latest value immediately

- Implementing state management patterns

Use ReplaySubject when:

- Late subscribers need historical events
- Caching values for new observers
- Event history is important

Use Observable (without Subject) when:

- Working with HTTP requests, timers, etc.
- Creating derived streams with operators
- Read-only data streams from services

4. Advanced Event Patterns

4.1 Event Delegation Pattern

Handle events from multiple child elements using a single event handler.

Component TypeScript:

```
export class ListComponent {
  items = [
    { id: 1, name: 'Item 1' },
    { id: 2, name: 'Item 2' },
    { id: 3, name: 'Item 3' }
  ];

  handleListClick(event: Event) {
    const target = event.target as HTMLElement;
```

```
// Find which item was clicked
if (target.classList.contains('item-delete')) {
  const id = target.getAttribute('data-id');
  this.deleteItem(Number(id));
} else if (target.classList.contains('item-edit')) {
  const id = target.getAttribute('data-id');
  this.editItem(Number(id));
}
}

deleteItem(id: number) {
  console.log('Delete item:', id);
  this.items = this.items.filter(item => item.id !== id);
}

editItem(id: number) {
  console.log('Edit item:', id);
}
}
```

Template:

```
<!-- Single event handler on parent -->
<ul (click)="handleListClick($event)">
  <li *ngFor="let item of items">
    {{ item.name }}
    <button class="item-edit" [attr.data-id]="item.id">Edit</button>
    <button class="item-delete" [attr.data-id]="item.id">Delete</button>
  </li>
</ul>
```

4.2 Debouncing Events

Delay event handling to reduce frequency of expensive operations.

Search Component with Debounce:

```
import { Component, Output, EventEmitter, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { debounceTime, distinctUntilChanged } from 'rxjs/operators';

@Component({
```

```
selector: 'app-search',
template: `
  <input
    [value]="searchTerm"
    (input)="onSearchInput($event)"
    placeholder="Search...">
`
})
export class SearchComponent implements OnDestroy {
  @Output() searchChanged = new EventEmitter<string>();

  searchTerm = '';
  private searchSubject = new Subject<string>();

  constructor() {
    // Debounce search input by 300ms
    this.searchSubject.pipe(
      debounceTime(300),
      distinctUntilChanged()
    ).subscribe(term => {
      this.searchChanged.emit(term);
    });
  }

  onSearchInput(event: Event) {
    const input = event.target as HTMLInputElement;
    this.searchTerm = input.value;

    // Push to subject instead of emitting directly
    this.searchSubject.next(this.searchTerm);
  }

  ngOnDestroy() {
    this.searchSubject.complete();
  }
}
```

Without Debounce:

```
User types: "a" → API call
User types: "n" → API call
User types: "g" → API call
User types: "u" → API call
User types: "l" → API call
User types: "a" → API call
```

User types: "r" → API call

Result: 7 API calls

With Debounce (300ms):

User types: "angular" (fast)

... waits 300ms ...

→ API call

Result: 1 API call

4.3 Throttling Events

Limit the rate of event handling.

Scroll Handler with Throttle:

```
import { Component, HostListener, OnInit, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { throttleTime } from 'rxjs/operators';

@Component({
  selector: 'app-infinite-scroll',
  template: `
    <div class="content">
      <!-- Long content -->
    </div>
    <div *ngIf="loading">Loading more...</div>
  `
})
export class InfiniteScrollComponent implements OnInit, OnDestroy {
  loading = false;
  private scrollSubject = new Subject<void>();

  ngOnInit() {
    // Throttle scroll events to max once per second
    this.scrollSubject.pipe(
      throttleTime(1000)
    ).subscribe(() => {
      this.loadMoreData();
    });
  }

  @HostListener('window:scroll', ['$event'])
  onScroll() {
```



```
const scrollTopPosition = window.pageYOffset + window.innerHeight;
const pageHeight = document.documentElement.scrollHeight;

if (scrollTopPosition >= pageHeight - 100) {
  // Push to subject instead of calling directly
  this.scrollSubject.next();
}

loadMoreData() {
  if (this.loading) return;

  this.loading = true;
  console.log('Loading more data...');

  // Simulate API call
  setTimeout(() => {
    this.loading = false;
  }, 1000);
}

ngOnDestroy() {
  this.scrollSubject.complete();
}
}
```

4.4 Combining Multiple Events

React to combinations of events using RxJS operators.

Form Validation Component:

```
import { Component, OnInit } from '@angular/core';
import { combineLatest, Subject, BehaviorSubject } from 'rxjs';
import { map } from 'rxjs/operators';

@Component({
  selector: 'app-form-validation',
  template: `
    <form>
      <input
        (input)="username$.next($any($event.target).value)"
        placeholder="Username">
    </form>
  `
})
export class FormValidationComponent implements OnInit {
  username$: Subject<string> = new Subject();
  ngOnInit(): void {
    // ...
  }
}
```

```

<input
  type="password"
  (input)="password$.next($any($event.target).value)"
  placeholder="Password">

<input
  type="checkbox"
  (change)="terms$.next($any($event.target).checked)">
  Accept terms

<button [disabled]="!(isFormValid$ | async)">Submit</button>
</form>

<div *ngIf="isFormValid$ | async">
  ✓ Form is valid
</div>
,
})
export class FormValidationComponent implements OnInit {
  username$ = new BehaviorSubject<string>('');
  password$ = new BehaviorSubject<string>('');
  terms$ = new BehaviorSubject<boolean>(false);

  isFormValid$: Observable<boolean>;

  ngOnInit() {
    // Combine all form field observables
    this.isFormValid$ = combineLatest([
      this.username$,
      this.password$,
      this.terms$
    ]).pipe(
      map(([username, password, terms]) => {
        return username.length >= 3 &&
          password.length >= 6 &&
            terms === true;
      })
    );
  }
}

```

Form Validation



Accept terms

[Submit Button - Enabled]

✓ Form is valid

Real-time Validation:

- Username: john (✓ >= 3 chars)
- Password: pass123 (✓ >= 6 chars)
- Terms: true (✓ accepted)
- Form Valid: true

4.5 Event Filtering and Transformation

Use RxJS operators to filter and transform events.

Click Counter with Filtering:

```
import { Component, OnInit } from '@angular/core';
import { fromEvent, Subject } from 'rxjs';
import { filter, map, scan, takeUntil } from 'rxjs/operators';

@Component({
  selector: 'app-click-counter',
  template: `
    <button #clickButton>Click Me</button>
    <p>Ctrl+Clicks: {{ ctrlClickCount }}</p>
    <p>Regular Clicks: {{ regularClickCount }}</p>
  `
})
export class ClickCounterComponent implements OnInit, OnDestroy {
  @ViewChild('clickButton', { static: true }) button!: ElementRef;

  ctrlClickCount = 0;
  regularClickCount = 0;

  private destroy$ = new Subject<void>();

  ngOnInit() {
    const clicks$ = fromEvent<MouseEvent>(
```

```
        this.button.nativeElement,  
        'click'  
    );  
  
    // Filter only Ctrl+Clicks  
    clicks$.pipe(  
        filter(event => event.ctrlKey),  
        scan(count => count + 1, 0),  
        takeUntil(this.destroy$)  
    ).subscribe(count => {  
        this.ctrlClickCount = count;  
    });  
  
    // Filter only regular clicks (no Ctrl)  
    clicks$.pipe(  
        filter(event => !event.ctrlKey),  
        scan(count => count + 1, 0),  
        takeUntil(this.destroy$)  
    ).subscribe(count => {  
        this.regularClickCount = count;  
    });  
}  
  
ngOnDestroy() {  
    this.destroy$.next();  
    this.destroy$.complete();  
}  
}
```

5. Custom Event Bus Implementation

5.1 Complete Event Bus Service

Advanced Event Bus (event-bus.service.ts):

```
import { Injectable } from '@angular/core';  
import { Subject, Observable, Subscription } from 'rxjs';  
import { filter, map } from 'rxjs/operators';  
  
export interface IEvent {  
    type: string;  
}
```

```
    payload?: any;
  }

  @Injectable({
    providedIn: 'root'
  })
  export class EventBusService {
    private subject$ = new Subject<IEvent>();

    /**
     * Emit an event
     */
    emit(event: IEvent): void {
      this.subject$.next(event);
    }

    /**
     * Subscribe to all events
     */
    on(eventType: string, handler: (payload: any) => void): Subscription {
      return this.subject$.pipe(
        filter(event => event.type === eventType),
        map(event => event.payload)
      ).subscribe(handler);
    }

    /**
     * Get observable for specific event type
     */
    observe(eventType: string): Observable<any> {
      return this.subject$.pipe(
        filter(event => event.type === eventType),
        map(event => event.payload)
      );
    }

    /**
     * Emit event with type and payload
     */
    dispatch(type: string, payload?: any): void {
      this.emit({ type, payload });
    }
  }
}
```

5.2 Event Types Constants

Event Types (event-types.ts):

```
export const EventTypes = {  
  // User events  
  USER_LOGGED_IN: 'USER_LOGGED_IN',  
  USER_LOGGED_OUT: 'USER_LOGGED_OUT',  
  USER_PROFILE_UPDATED: 'USER_PROFILE_UPDATED',  
  
  // Data events  
  DATA_LOADED: 'DATA_LOADED',  
  DATA_SAVED: 'DATA_SAVED',  
  DATA_DELETED: 'DATA_DELETED',  
  
  // UI events  
  MODAL_OPENED: 'MODAL_OPENED',  
  MODAL_CLOSED: 'MODAL_CLOSED',  
  NOTIFICATION_SHOWN: 'NOTIFICATION_SHOWN',  
  
  // Navigation events  
  ROUTE_CHANGED: 'ROUTE_CHANGED',  
  PAGE_LOADED: 'PAGE_LOADED'  
} as const;  
  
export type EventType = typeof EventTypes[keyof typeof EventTypes];
```

5.3 Using the Event Bus

Login Component - Emitting Events:

```
import { Component } from '@angular/core';  
import { EventBusService } from './event-bus.service';  
import { EventTypes } from './event-types';  
  
@Component({  
  selector: 'app-login',  
  template: `  
    <button (click)="login()">Login</button>  
  `,  
})  
export class LoginComponent {  
  constructor(private eventBus: EventBusService) {}  
}
```

```
login() {  
  // Simulate login  
  const user = {  
    id: 1,  
    name: 'John Doe',  
    email: 'john@example.com'  
  };  
  
  // Emit login event  
  this.eventBus.dispatch(EventTypes.USER_LOGGED_IN, user);  
}  
}
```

Header Component - Listening to Events:

```
import { Component, OnInit, OnDestroy } from '@angular/core';  
import { Subscription } from 'rxjs';  
  
@Component({  
  selector: 'app-header',  
  template: `  
    <div class="header">  
      <span *ngIf="currentUser">Welcome, {{ currentUser.name }}</span>  
    </div>  
  `,  
})  
export class HeaderComponent implements OnInit, OnDestroy {  
  currentUser: any = null;  
  private subscription!: Subscription;  
  
  constructor(private eventBus: EventBusService) {}  
  
  ngOnInit() {  
    // Listen to login events  
    this.subscription = this.eventBus.on(  
      EventTypes.USER_LOGGED_IN,  
      (user) => {  
        this.currentUser = user;  
        console.log('User logged in:', user);  
      }  
    );  
  }  
  
  ngOnDestroy() {
```

```
    if (this.subscription) {  
      this.subscription.unsubscribe();  
    }  
  }  
}
```

Multiple Components Listening:

```
// Analytics Component  
export class AnalyticsComponent implements OnInit {  
  ngOnInit() {  
    this.eventBus.on(EventTypes.USER_LOGGED_IN, (user) => {  
      this.trackLogin(user);  
    });  
  }  
  
  trackLogin(user: any) {  
    console.log('Tracking login for:', user.email);  
  }  
}  
  
// Notification Component  
export class NotificationComponent implements OnInit {  
  ngOnInit() {  
    this.eventBus.on(EventTypes.USER_LOGGED_IN, (user) => {  
      this.showWelcomeNotification(user);  
    });  
  }  
  
  showWelcomeNotification(user: any) {  
    console.log(`Welcome ${user.name}!`);  
  }  
}  
  
// Dashboard Component  
export class DashboardComponent implements OnInit {  
  ngOnInit() {  
    this.eventBus.on(EventTypes.USER_LOGGED_IN, (user) => {  
      this.loadUserDashboard(user);  
    });  
  }  
  
  loadUserDashboard(user: any) {
```



```
    console.log('Loading dashboard for:', user.id);  
  }  
}
```

Event Flow When User Logs In:

1. LoginComponent emits: USER_LOGGED_IN
Payload: {id: 1, name: "John Doe", email: "john@example.com"}
 2. HeaderComponent receives event
Console: User logged in: {id: 1, name: "John Doe", ...}
Updates UI: "Welcome, John Doe"
 3. AnalyticsComponent receives event
Console: Tracking login for: john@example.com
 4. NotificationComponent receives event
Console: Welcome John Doe!
 5. DashboardComponent receives event
Console: Loading dashboard for: 1
- All components react to single event automatically!**

6. Memory Management and Cleanup

6.1 Subscription Leaks

Failing to unsubscribe from observables causes memory leaks.

✗ Bad Practice - Memory Leak:

```
export class BadComponent implements OnInit {  
  constructor(private dataService: DataService) {}  
  
  ngOnInit() {  
    // Subscription never cleaned up!  
    this.dataService.getData().subscribe(data => {
```

```
        console.log(data);
    });
}

// Component destroyed but subscription still active = MEMORY LEAK
}
```

6.2 Manual Unsubscription

☒ Good Practice - Manual Cleanup:

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subscription } from 'rxjs';

export class GoodComponent implements OnInit, OnDestroy {
    private subscription!: Subscription;

    constructor(private dataService: DataService) {}

    ngOnInit() {
        this.subscription = this.dataService.getData().subscribe(data => {
            console.log(data);
        });
    }

    ngOnDestroy() {
        // Clean up subscription
        if (this.subscription) {
            this.subscription.unsubscribe();
        }
    }
}
```

6.3 Multiple Subscriptions

☒ Managing Multiple Subscriptions:

```
export class MultiSubscriptionComponent implements OnInit, OnDestroy {
    private subscriptions = new Subscription();

    ngOnInit() {
```

```
// Add multiple subscriptions to single container
this.subscriptions.add(
  this.service1.data$.subscribe(data => { })
);

this.subscriptions.add(
  this.service2.events$.subscribe(event => { })
);

this.subscriptions.add(
  this.service3.updates$.subscribe(update => { })
);
}

ngOnDestroy() {
  // Unsubscribe from all at once
  this.subscriptions.unsubscribe();
}
}
```

6.4 Using takeUntil Pattern

☒ Best Practice - takeUntil Operator:

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

export class BestPracticeComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    // All subscriptions automatically complete when destroy$ emits
    this.service1.data$.pipe(
      takeUntil(this.destroy$)
    ).subscribe(data => {
      console.log(data);
    });

    this.service2.events$.pipe(
      takeUntil(this.destroy$)
    ).subscribe(event => {
      console.log(event);
    });
  }
}
```

```
this.service3.updates$.pipe(
  takeUntil(this.destroy$)
).subscribe(update => {
  console.log(update);
});
}

ngOnDestroy() {
  // Emit to complete all subscriptions
  this.destroy$.next();
  this.destroy$.complete();
}
}
```

6.5 Async Pipe - No Manual Unsubscription Needed

☒ Automatic Cleanup with Async Pipe:

```
@Component({
  template: `
    <!-- Async pipe automatically subscribes and unsubscribes -->
    <div *ngFor="let item of data$ | async">
      {{ item.name }}
    </div>

    <p>User: {{ user$ | async | json }}</p>
  `
})
export class AsyncPipeComponent {
  data$: Observable<any[]>;
  user$: Observable<User>;

  constructor(private dataService: DataService) {
    this.data$ = this.dataService.getData();
    this.user$ = this.dataService.getUser();
  }

  // No ngOnDestroy needed! Async pipe handles cleanup
}
```

Subscription Cleanup Priority:

1. **Best:** Use async pipe in template (automatic cleanup)
2. **Good:** Use takeUntil pattern for programmatic subscriptions
3. **Acceptable:** Store subscriptions and unsubscribe in ngOnDestroy
4. **Bad:** No cleanup (memory leak)

7. Real-World Example: Chat Application

7.1 Chat Service with Events

Chat Service (chat.service.ts):

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Subject, Observable } from 'rxjs';

export interface Message {
  id: string;
  text: string;
  sender: string;
  timestamp: Date;
  type: 'sent' | 'received';
}

export interface TypingStatus {
  userId: string;
  isTyping: boolean;
}

@Injectable({
  providedIn: 'root'
})
export class ChatService {
  // Messages state
  private messagesSubject = new BehaviorSubject<Message[]>([]);
  public messages$ = this.messagesSubject.asObservable();

  // New message event
```

```
private newMessageSubject = new Subject<Message>();
public newMessage$ = this.newMessageSubject.asObservable();

// Typing status
private typingSubject = new Subject<TypingStatus>();
public typing$ = this.typingSubject.asObservable();

// Online users
private onlineUsersSubject = new BehaviorSubject<string[]>([]);
public onlineUsers$ = this.onlineUsersSubject.asObservable();

sendMessage(text: string, sender: string) {
  const message: Message = {
    id: Date.now().toString(),
    text,
    sender,
    timestamp: new Date(),
    type: 'sent'
  };

  // Add to messages list
  const currentMessages = this.messagesSubject.value;
  this.messagesSubject.next([...currentMessages, message]);

  // Emit new message event
  this.newMessageSubject.next(message);

  // Simulate receiving response
  setTimeout(() => {
    this.receiveMessage('This is an automated response');
  }, 1000);
}

receiveMessage(text: string) {
  const message: Message = {
    id: Date.now().toString(),
    text,
    sender: 'Bot',
    timestamp: new Date(),
    type: 'received'
  };

  const currentMessages = this.messagesSubject.value;
  this.messagesSubject.next([...currentMessages, message]);
  this.newMessageSubject.next(message);
}
```

```
setTyping(userId: string, isTyping: boolean) {
  this.typingSubject.next({ userId, isTyping });
}

updateOnlineUsers(users: string[]) {
  this.onlineUsersSubject.next(users);
}

clearMessages() {
  this.messagesSubject.next([]);
}
}
```

7.2 Chat Component

Chat Component (chat.component.ts):

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { takeUntil, debounceTime } from 'rxjs/operators';

@Component({
  selector: 'app-chat',
  template: `
    <div class="chat-container">
      <!-- Header -->
      <div class="chat-header">
        <h3>Chat Room</h3>
        <span class="online-count">
          {{ onlineUsers.length }} online
        </span>
      </div>

      <!-- Messages -->
      <div class="messages" #messagesContainer>
        <div *ngFor="let msg of messages"
          [class]='message ' + msg.type">
          <div class="message-header">
            <strong>{{ msg.sender }}</strong>
            <small>{{ msg.timestamp | date:'short' }}</small>
          </div>
          <p>{{ msg.text }}</p>
        </div>
      </div>
    </div>`
})
export class ChatComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject();
  typingSubject = new Subject();
  onlineUsersSubject = new Subject();
  messagesSubject = new Subject();

  ngOnInit() {
    // ...
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

```

    <div *ngIf="someoneTyping" class="typing-indicator">
      Someone is typing...
    </div>
  </div>

  <!-- Input -->
  <div class="chat-input">
    <input
      [(ngModel)]="messageText"
      (ngModelChange)="onTyping()"
      (keydown.enter)="sendMessage()"
      placeholder="Type a message...">
    <button (click)="sendMessage()">Send</button>
  </div>
</div>
`,
styles: [`
  .chat-container {
    border: 1px solid #ddd;
    border-radius: 8px;
    height: 500px;
    display: flex;
    flex-direction: column;
  }
  .chat-header {
    padding: 16px;
    background: #f5f5f5;
    border-bottom: 1px solid #ddd;
    display: flex;
    justify-content: space-between;
  }
  .messages {
    flex: 1;
    overflow-y: auto;
    padding: 16px;
  }
  .message {
    margin-bottom: 16px;
    padding: 12px;
    border-radius: 8px;
    max-width: 70%;
  }
  .message.sent {
    background: #007bff;
    color: white;
    margin-left: auto;
  }
]

```



```

.message.received {
  background: #e9ecef;
}
.typing-indicator {
  color: #6c757d;
  font-style: italic;
}
.chat-input {
  display: flex;
  padding: 16px;
  border-top: 1px solid #ddd;
}
.chat-input input {
  flex: 1;
  padding: 8px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
}
.chat-input button {
  margin-left: 8px;
  padding: 8px 16px;
}
`]
})
export class ChatComponent implements OnInit, OnDestroy {
  messages: Message[] = [];
  messageText = '';
  onlineUsers: string[] = [];
  someoneTyping = false;
  currentUser = 'You';

  private destroy$ = new Subject<void>();
  private typingSubject = new Subject<void>();

  constructor(private chatService: ChatService) {}

  ngOnInit() {
    // Subscribe to messages
    this.chatService.messages$.pipe(
      takeUntil(this.destroy$)
    ).subscribe(messages => {
      this.messages = messages;
      this.scrollToBottom();
    });

    // Subscribe to new message events
    this.chatService.newMessage$.pipe(

```

```
        takeUntil(this.destroy$)
    ).subscribe(message => {
        console.log('New message:', message);
        this.playNotificationSound();
    });

    // Subscribe to typing status
    this.chatService.typing$.pipe(
        takeUntil(this.destroy$)
    ).subscribe(status => {
        this.someoneTyping = status.isTyping;
    });

    // Subscribe to online users
    this.chatService.onlineUsers$.pipe(
        takeUntil(this.destroy$)
    ).subscribe(users => {
        this.onlineUsers = users;
    });

    // Debounce typing indicator
    this.typingSubject.pipe(
        debounceTime(1000),
        takeUntil(this.destroy$)
    ).subscribe(() => {
        this.chatService.setTyping(this.currentUser, false);
    });

    // Simulate online users
    this.chatService.updateOnlineUsers(['You', 'Bot', 'User3']);
}

sendMessage() {
    if (this.messageText.trim()) {
        this.chatService.sendMessage(this.messageText, this.currentUser);
        this.messageText = '';
        this.chatService.setTyping(this.currentUser, false);
    }
}

onTyping() {
    this.chatService.setTyping(this.currentUser, true);
    this.typingSubject.next();
}

playNotificationSound() {
    console.log('🔔 Notification sound');
```

```

}

scrollToBottom() {
  // Scroll logic
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
}

```

Chat Room (3 online)

Bot 1/30/26, 2:25 PM
Welcome to the chat!

You 1/30/26, 2:26 PM
Hello, everyone!

Bot 1/30/26, 2:26 PM
This is an automated response

Someone is typing...

Send

Console Output:

New message: {id: "1738242360000", text: "Hello, everyone!", ...}

🔔 Notification sound

New message: {id: "1738242361000", text: "This is an automated response", ...}

🔔 Notification sound

8. Best Practices Summary

8.1 EventEmitter Guidelines

Do:

- Use EventEmitter only with @Output decorator
- Always type your EventEmitter: `EventEmitter<Type>`
- Name events with action verbs: `itemDeleted` , `formSubmitted`
- Complete EventEmitter in ngOnDestroy if needed
- Emit immutable objects (use spread operator or Object.assign)

Don't:

- Don't use EventEmitter in services (use Subject instead)
- Don't mutate emitted objects after emission
- Don't forget to type EventEmitter
- Don't emit events in constructor
- Don't create complex event chains (hard to debug)

8.2 Subject/Observable Guidelines

Do:

- Use Subject for event buses and multicasting
- Use BehaviorSubject for state management
- Expose Observables (`asObservable()`) to consumers
- Always unsubscribe or use `takeUntil`

- Use async pipe when possible
- Complete Subjects in ngOnDestroy

Don't:

- Don't expose Subjects directly (use asObservable())
- Don't forget to unsubscribe
- Don't use Subjects for simple one-time values
- Don't create subscription chains without cleanup

8.3 Performance Tips

1. **Debounce rapid events:** Use debounceTime for search, scroll
2. **Throttle high-frequency events:** Use throttleTime for scroll, resize
3. **Unsubscribe properly:** Prevent memory leaks
4. **Use event delegation:** Single handler for multiple elements
5. **Avoid complex event chains:** Keep event flow simple

9. Practice Exercises

Exercise 1: Build a Todo List with Events

Create a todo application with:

- TodoItem component that emits events for edit, delete, complete
- TodoList component that handles all events
- Use EventEmitter for component communication
- Implement undo functionality using event history

Exercise 2: Implement Real-Time Search

Build a search component that:

- Debounces user input (300ms delay)
- Emits search events only when term length ≥ 3
- Shows loading indicator during search
- Cancels previous search if new one starts
- Uses Subject for debouncing

Exercise 3: Create a Notification System

Implement a notification service with:

- BehaviorSubject for notification state
- Multiple components can trigger notifications
- Notification component displays all notifications
- Auto-dismiss after 5 seconds
- Support for different notification types (info, success, error)

Exercise 4: Build a Shopping Cart with Event Bus

Create a shopping cart system where:

- Product components emit "add to cart" events to event bus
- Cart component listens to all cart events
- Header shows cart count (listens to cart updates)
- Checkout component listens to cart changes
- Use custom event bus service

Exercise 5: Form Validation with Combined Observables

Build a registration form that:

- Uses BehaviorSubject for each form field
- Combines all fields with combineLatest

- Validates in real-time using operators
- Disables submit button until form is valid
- Shows field-specific error messages

10. Summary

In this module, you learned:

- **EventEmitter:** Angular's built-in class for component events with @Output
- **Custom Events:** Creating and handling custom component events
- **Subject Types:** Subject, BehaviorSubject, ReplaySubject for different use cases
- **Event Bus Pattern:** Cross-component communication without direct coupling
- **Advanced Patterns:** Debouncing, throttling, filtering, and combining events
- **Memory Management:** Proper subscription cleanup with takeUntil and async pipe
- **Best Practices:** When to use each event type and how to avoid common pitfalls

Event handling is crucial for building interactive Angular applications. Understanding the differences between EventEmitter, Subject, and Observable helps you choose the right tool for each scenario.

Next Module Preview: In Module 8.3, we'll explore Service-Based Communication patterns, including BehaviorSubject for state management and strategies for unrelated component communication.

Module 8.2 - Event Handling | Angular Framework Training